



# Aula 10: Conectando Node JS ao MongoDB

⚙ Status Não iniciada

## Aula 10 — Conectando Node.js ao MongoDB

### Objetivo da aula

Nesta aula, você vai conectar sua API Node.js a um banco de dados MongoDB.

Ao final, você vai saber:

- ✓ criar um banco no MongoDB Atlas
- ✓ instalar o Mongoose
- ✓ usar variável de ambiente com `.env`
- ✓ conectar Node.js ao MongoDB
- ✓ criar o primeiro schema
- ✓ criar o primeiro model
- ✓ testar se o banco conectou corretamente

---

# 1. O que vamos fazer nesta aula? 🧠

Até agora, sua API guarda produtos em um array:

```
let produtos = [  
  {  
    id: 1,  
    nome: "Mouse Gamer",  
    preco: 120  
  }  
];
```

O problema é que, quando o servidor reinicia, os dados somem.

Agora vamos começar a salvar os dados em um banco real.

Nesta aula, você vai preparar a conexão com o MongoDB.

Na próxima aula, você vai trocar o CRUD em memória por CRUD com banco de dados.

---

## 3. Criando uma conta no MongoDB Atlas

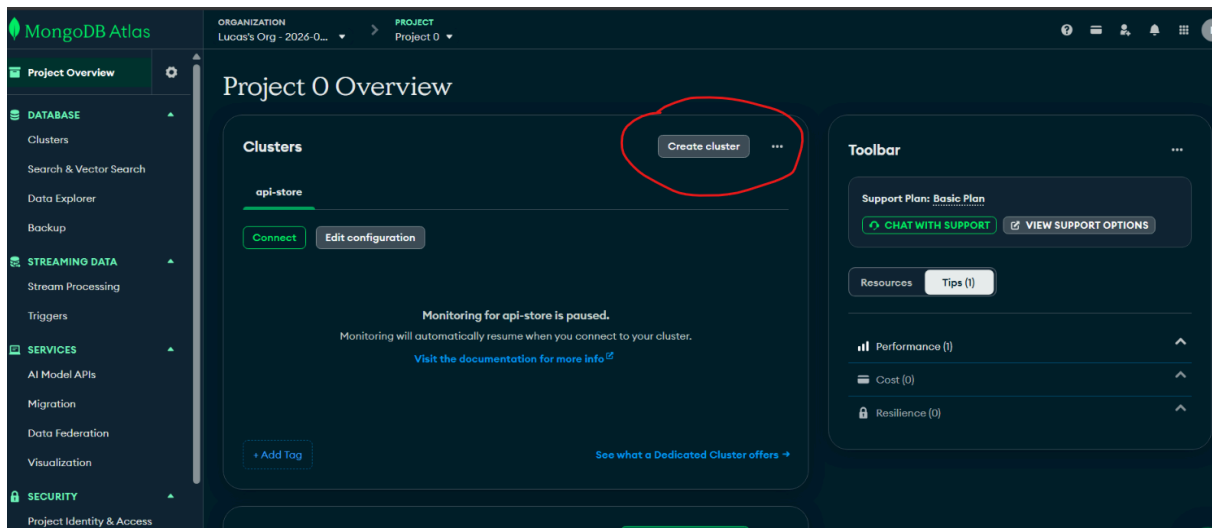


Acesse o site do MongoDB Atlas e crie uma conta.

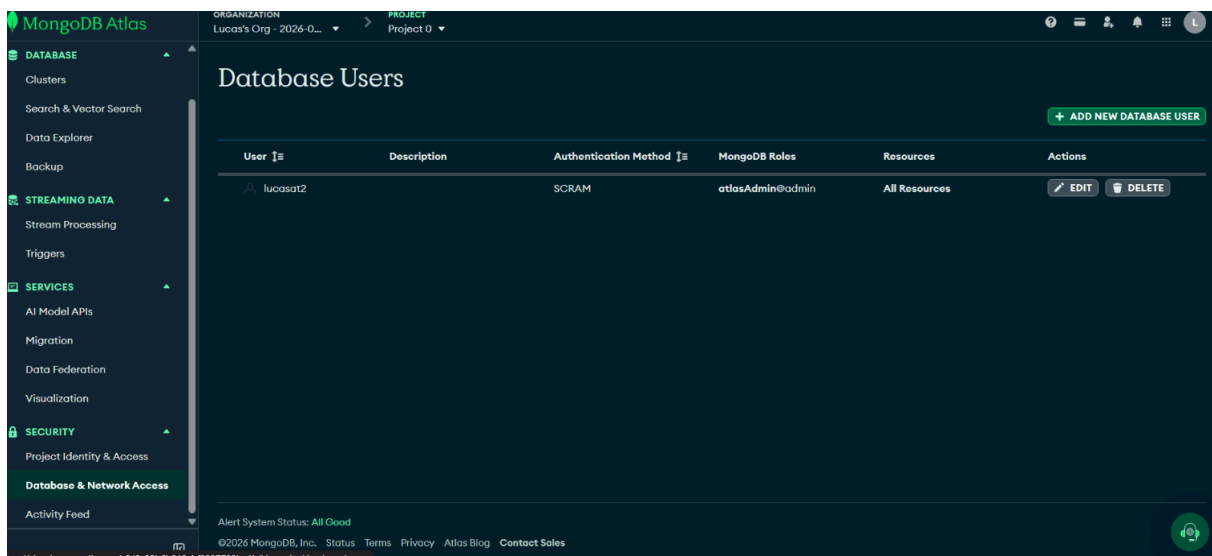
Depois siga este fluxo:

### 0 - Criar uma conta (ou logar com google)

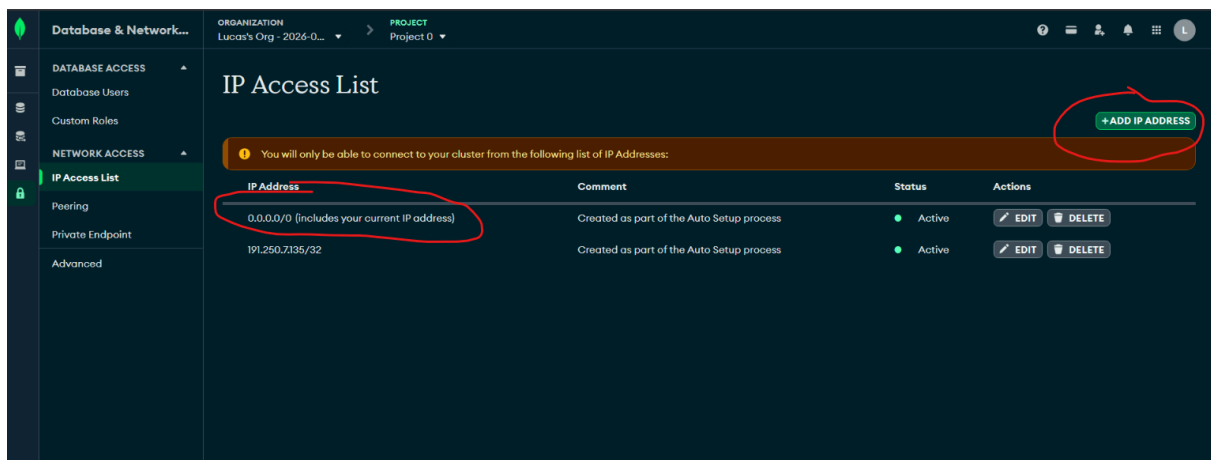
#### 1- Criar um Cluster( Servidor)



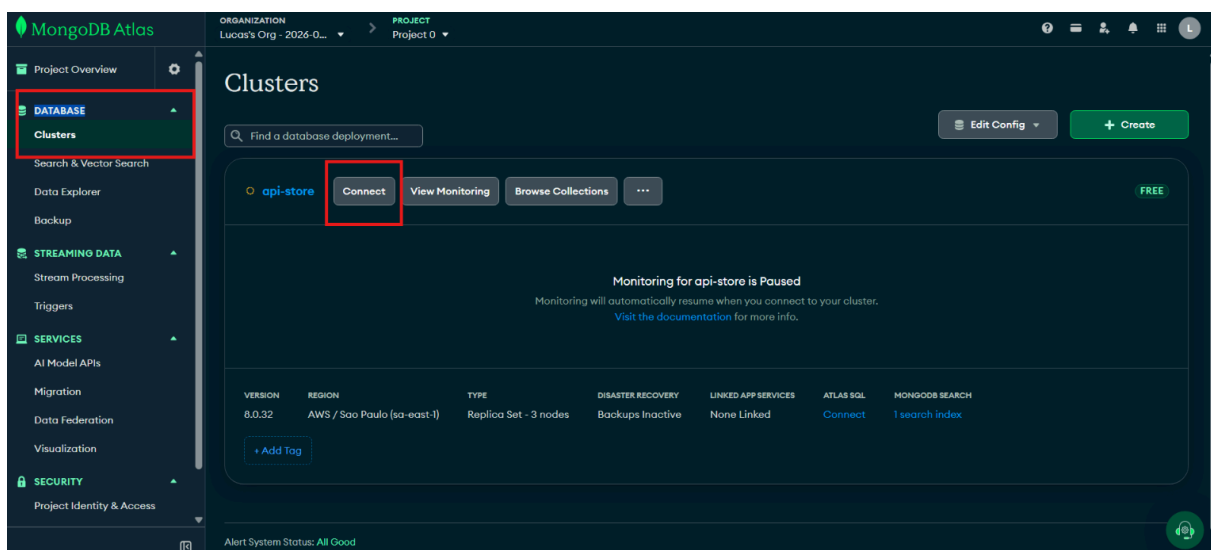
## 2 - Criar um usuario



## 3 - Liberar Ip



## 4 - Pegar as configurações de conexão ao banco



PROJECT

Connect to api-store

1

Set up connection security

2

Choose a connection method

3

Connect

Connect to your application



Drivers

Access your Atlas data using MongoDB's native drivers (e.g. Node.js, Go, etc.)

>

Access your data through tools



Compass

Explore, modify, and visualize your data with MongoDB's GUI

>



Shell

Quickly add & update data using MongoDB's Javascript command-line interface

>



MongoDB for VS Code

Work with your data in MongoDB directly from your VS Code environment

>



Atlas SQL

>

✓

✓

3

Set up connection security

Choose a connection method

Connect

### Connecting with MongoDB Driver

#### 1. Select your driver and version

We recommend installing and using the latest driver version.

| Driver  | Version      |
|---------|--------------|
| Node.js | 6.7 or later |

#### 2. Install your driver

Run the following on the command line

```
npm install mongodb
```

[View MongoDB Node.js Driver installation instructions.](#)

#### 3. Add your connection string into your application code

Use this connection string in your application

☒ SRV Connection String ? ☐ View full code sample

```
mongodb+srv://lucasat2:<db_password>@api-store.reoni4j.mongodb.net/?appName=api-store
```

Replace **<db\_password>** with the password for the **lucasat2** database user. Ensure any option params are [URL encoded](#).

O cluster é o servidor onde seu banco vai ficar hospedado.

Você pode pensar assim:

```
Cluster = servidor do banco
Database = banco específico
Collection = conjunto de documentos
Document = item salvo
```

E também pode colocar o nome do banco depois de `.net/`.

Exemplo:

```
mongodb+srv://admin:minhaSenha123@cluster0.xxxxx.mongodb.net/api-store?retryWrites=true&w=majority
```

Nesse exemplo, o banco se chama:

## 8. Instalando o Mongoose

Agora volte para seu projeto Node.js.

No terminal, instale o Mongoose:

```
npm install mongoose
```

O Mongoose é a biblioteca que vamos usar para conversar com o MongoDB.

Com ele, você consegue:

- ✓ conectar no banco
- ✓ criar schemas
- ✓ criar models
- ✓ salvar dados
- ✓ buscar dados
- ✓ atualizar dados
- ✓ deletar dados

## 9. Instalando o dotenv

Agora instale o `dotenv`:

```
npm install dotenv
```

O `dotenv` permite usar variáveis de ambiente.

Isso é importante porque você não deve colocar a string do banco diretamente no código.

Errado:

```
mongoose.connect("mongodb+srv://admin:senha123@cluste
```

```
r..."");
```

Assim, informações sensíveis ficam em um arquivo `.env`.

## 10. Criando o arquivo `.env`

Na raiz do projeto, crie um arquivo chamado:

```
.env
```

Dentro dele, coloque:

```
PORT=3000  
MONGO_URL=sua_string_de_conexao_aqui
```

Exemplo:

```
PORT=3000  
MONGO_URL=mongodb+srv://admin:minhaSenha123@cluster0.xxxxx.mongodb.net/api-store?retryWrites=true&w=majority
```

 O arquivo `.env` não deve ser enviado para o GitHub.

## 11. Criando o `.gitignore`

Na raiz do projeto, crie o arquivo:

```
.gitignore
```

Adicione:

```
node_modules  
.env
```

Isso evita que você envie para o GitHub:

- dependências pesadas

- senha do banco
- string de conexão

Esse cuidado é muito importante no mercado.

---

## 12. Criando a pasta de configuração

Dentro da pasta `src`, crie:

```
config
```

Dentro dela, crie o arquivo:

```
database.js
```

A estrutura ficará assim:

```
src/  
  config/  
    database.js  
  controllers/  
  routes/  
  services/  
  server.js
```

## 13. Criando a conexão com o banco

No arquivo:

```
src/config/database.js
```

adicione:

```
const mongoose = require("mongoose");  
  
async function connectDatabase() {  
  try {
```

```
await mongoose.connect(process.env.MONGO_URL);

console.log("MongoDB conectado com sucesso.");
} catch (error) {
  console.error("Erro ao conectar no MongoDB:", error.message);
  process.exit(1);
}
}

module.exports = connectDatabase;
```

## 14. Entendendo o código 🧠

### Importando o Mongoose

```
const mongoose = require("mongoose");
```

Aqui você importa a biblioteca que conversa com o MongoDB.

### Criando função assíncrona

```
async function connectDatabase() {
```

Conectar no banco é uma operação assíncrona.

Por isso usamos `async`.

### Esperando a conexão

```
await mongoose.connect(process.env.MONGO_URL);
```

Aqui o Node.js tenta conectar no banco usando a string salva no `.env`.

### Tratando erro

```
catch (error) {  
  console.error("Erro ao conectar no MongoDB:", error.message);  
  process.exit(1);  
}
```

Se a conexão falhar, o erro aparece no terminal e o servidor é encerrado. Isso evita que sua API rode sem banco.

## 15. Atualizando o `server.js`

Agora abra:

```
src/server.js
```

Importe o `dotenv` e a conexão com o banco:

```
require("dotenv").config();  
  
const express = require("express");  
  
const connectDatabase = require("./config/database");  
const productRoutes = require("./routes/productRoutes");  
const logger = require("./middlewares/loggerMiddleware");  
const errorHandler = require("./middlewares/errorMiddleware");  
  
const app = express();  
  
connectDatabase();  
  
app.use(express.json());  
  
app.use(logger);
```

```
app.use(productRoutes);

app.use((req, res) => {
  res.status(404).json({
    erro: "Rota não encontrada."
  });
});

app.use(errorHandler);

const PORT = process.env.PORT || 3000;

app.listen(PORT, () => {
  console.log(`Servidor rodando na porta ${PORT}`);
});
```

## 16. Entendendo o novo `server.js`

Esta linha carrega as variáveis do `.env`:

```
require("dotenv").config();
```

Esta linha conecta no banco:

```
connectDatabase();
```

Esta linha usa a porta do `.env`:

```
const PORT = process.env.PORT || 3000;
```

Se não existir `PORT` no `.env`, ele usa `3000`.

## 17. Rodando o servidor

No terminal, execute:

```
npm run dev
```

Ou:

```
node src/server.js
```

Se tudo deu certo, você verá:

```
MongoDB conectado com sucesso.  
Servidor rodando na porta 3000
```

Se der erro, confira:

- sua internet
- string de conexão
- usuário do banco
- senha do banco
- liberação de IP
- nome correto da variável `MONGO_URL`

## 18. Criando a pasta de models

Agora vamos criar nosso primeiro model.

Dentro de `src`, crie a pasta:

```
models
```

Dentro dela, crie o arquivo:

```
Product.js
```

Estrutura:

```
src/  
  config/  
    database.js  
  models/  
    Product.js  
  routes/  
  controllers/  
  services/  
  server.js
```

## 19. Criando o schema de Produto

No arquivo:

```
src/models/Product.js
```

adicione:

```
const mongoose = require("mongoose");  
  
const productSchema = new mongoose.Schema({  
  nome: {  
    type: String,  
    required: true  
  },  
  preco: {  
    type: Number,  
    required: true  
  },  
  categoria: {  
    type: String,  
    required: true  
  },  
  estoque: {  
    type: Number,  
    required: true  
  }  
});
```

```
const Product = mongoose.model("Product", productSchema);

module.exports = Product;
```

## 20. Entendendo o schema

O schema define como o produto deve ser salvo no banco.

Neste caso, todo produto precisa ter:

```
nome
preco
categoria
estoque
```

Cada campo tem um tipo:

```
nome = String
preco = Number
categoria = String
estoque = Number
```

E todos são obrigatórios:

```
required: true
```

## 21. O que é model?

O model é o recurso que você usa para conversar com a collection no banco.

Aqui:

```
const Product = mongoose.model("Product", productSchema);
```

Você criou um model chamado `Product`.

Com ele, você poderá usar:

```
Product.create()
Product.find()
Product.findById()
Product.findByIdAndUpdate()
Product.findByIdAndDelete()
```

Na próxima aula, você vai usar esses métodos para criar o CRUD real.

## 22. Testando se o model funciona

Para testar rapidamente, você pode criar uma rota temporária.

No `server.js`, antes da rota não encontrada, adicione:

```
const Product = require("./models/Product");

app.post("/teste-produto", async (req, res) => {
  const produto = await Product.create({
    nome: "Mouse Gamer",
    preco: 120,
    categoria: "Informática",
    estoque: 10
  });

  res.status(201).json(produto);
});
```

Agora teste no Postman:

```
POST http://localhost:3000/teste-produto
```

Se funcionar, o produto será salvo no MongoDB.

Depois do teste, você pode apagar essa rota temporária.

## 23. Verificando no MongoDB Atlas

Depois de criar o produto, vá ao MongoDB Atlas.

Procure por:

```
Browse Collections
```

Você deve encontrar:

```
Database: api-store  
Collection: products
```

Dentro dela, estará o produto criado.

O MongoDB cria a collection automaticamente.

## 24. Observação importante sobre nomes



Você criou o model:

```
mongoose.model("Product", productSchema);
```

O MongoDB normalmente cria a collection com o nome no plural e minúsculo:

```
products
```

Isso é normal.

Não precisa se preocupar agora.

## 25. Código final da aula

```
.env
```

```
PORT=3000  
MONGO_URL=sua_string_de_conexao_aqui
```

## **.gitignore**

```
node_modules
.env
```

## **src/config/database.js**

```
const mongoose = require("mongoose");

async function connectDatabase() {
  try {
    await mongoose.connect(process.env.MONGO_URL);

    console.log("MongoDB conectado com sucesso.");
  } catch (error) {
    console.error("Erro ao conectar no MongoDB:", error.message);
    process.exit(1);
  }
}

module.exports = connectDatabase;
```

## **src/models/Product.js**

```
const mongoose = require("mongoose");

const productSchema = new mongoose.Schema({
  nome: {
    type: String,
    required: true
  },
  preco: {
    type: Number,
    required: true
  },
});
```

```

    categoria: {
      type: String,
      required: true
    },
    estoque: {
      type: Number,
      required: true
    }
  });

const Product = mongoose.model("Product", productSchema);

module.exports = Product;

```

## src/server.js

```

require("dotenv").config();

const express = require("express");

const connectDatabase = require("./config/database");
const productRoutes = require("./routes/productRoutes");
const logger = require("./middlewares/loggerMiddleware");
const errorHandler = require("./middlewares/errorMiddleware");

const app = express();

connectDatabase();

app.use(express.json());
app.use(logger);

app.use(productRoutes);

app.use((req, res) => {
  res.status(404).json({

```

```
    erro: "Rota não encontrada."
  });
});

app.use(errorHandler);

const PORT = process.env.PORT || 3000;

app.listen(PORT, () => {
  console.log(`Servidor rodando na porta ${PORT}`);
});
```

## 26. Resumo da aula

Nesta aula, você aprendeu:

- ✓ MongoDB Atlas permite criar banco na nuvem
- ✓ Mongoose conecta Node.js ao MongoDB
- ✓ `dotenv` guarda variáveis de ambiente
- ✓ `.env` protege dados sensíveis
- ✓ `.gitignore` evita subir arquivos indevidos
- ✓ `mongoose.connect()` conecta ao banco
- ✓ `schema` define o formato dos dados
- ✓ `model` permite acessar a collection pelo código

## Exercícios

### Exercício 1 — Criar conexão com MongoDB

Crie o arquivo:

```
src/config/database.js
```

Dentro dele, crie a função `connectDatabase`.

Ela deve:

- usar `mongoose.connect`
- pegar a string do `process.env.MONGO_URL`
- mostrar mensagem de sucesso
- mostrar erro se falhar

## Exercício 2 — Criar model de aluno 🎓

Crie o arquivo:

```
src/models/Student.js
```

O schema deve ter:

```
nome: String, obrigatório  
email: String, obrigatório  
curso: String, obrigatório  
ativo: Boolean, obrigatório
```

Crie e exporte o model:

```
Student
```

## Exercício 3 — Criar model de pedido 🛒

Crie o arquivo:

```
src/models/Order.js
```

O schema deve ter:

```
cliente: String, obrigatório  
produto: String, obrigatório  
quantidade: Number, obrigatório
```

```
total: Number, obrigatório  
status: String, obrigatório
```

Crie e exporte o model:

```
Order
```